Practical Assignment No. 1

Transport Protocols and Streaming for Multimedia Services

The assignment must be completed in groups of three students. The estimated duration is 2.5 practical classes, complemented by independent group work during the specified period.

The assignment uses CORE Emulator as the experimental testbed. The working environment and resources required for the experiments will be made available on the e-learning platform.

1. Objectives

This assignment explores and compares communication mechanisms used to deliver multimedia services over IP networks.

Using CORE as the experimental testbed, students will analyse different solutions for streaming and transporting multimedia content, including adaptive streaming over HTTP, HTTP/3 over QUIC, and real-time multimedia communication.

The aim is to assess how network conditions, particularly available bandwidth, delay, and packet loss, affect application and protocol behaviour and the quality of the service delivered to users.

Throughout the assignment, students must collect and analyse traffic data, compare the solutions, and relate their findings to the characteristics of the underlying protocols.

By the end of the assignment, students should be able to:

- identify the main mechanisms used to distribute multimedia content over IP networks;
- analyse how adaptive streaming solutions respond to changing network conditions;
- understand the main differences between transport over TCP and over QUIC;
- analyse RTP/RTCP operation in real-time multimedia communication;
- compare unicast and multicast in terms of generated traffic and scalability;
- use traffic monitoring and analysis tools to support experimental conclusions;
- **measure, compare, and interpret the behaviour of the solutions studied.**

2. Tools

The main tools used in this assignment are:

- CORE Emulator — network infrastructure emulation;
- tcpdump — traffic capture on topology nodes;
- Wireshark — capture analysis and graph generation;
- FFmpeg / ffprobe — multimedia content analysis and processing;
- MPEG-DASH / DASH player — adaptive streaming;

© Flávio de Oliveira Silva - Department of Informatics, University of Minho – flavio@di.uminho.pt – For academic use only. External redistribution is prohibited.

- nginx — server used in the HTTP/2 experiments;
- Caddy — server used in the HTTP/3 experiments;
- HTTP/2/TCP and HTTP/3/QUIC client/server applications — provided for the experiments;
- other auxiliary tools specified in the instructions.

The goal is to use these tools to observe, measure, and understand service and protocol behaviour, rather than to learn the tools for their own sake.

3. Overview

The assignment consists of initial preparation followed by three experimental stages.

- During preparation, students will validate the working environment and analyse the multimedia content used in the experiments. Where possible, the content, configurations, and auxiliary applications will be provided in advance to reduce time spent on operational tasks.
- Stage 1 examines adaptive streaming over HTTP using MPEG-DASH. Different representations of the same content will be used while network conditions are changed in a controlled manner to observe application adaptation and relate the selected representation to the resources actually available.
- Stage 2 examines content transport using HTTP/3 over QUIC and compares it with an HTTP solution based on TCP. Experiments under different delays, packet losses, and available bandwidths aim to identify and interpret relevant differences between the approaches.
- Stage 3 examines real-time multimedia transmission using RTP/RTCP over UDP, comparing unicast and multicast. Students will assess how increasing the number of clients affects network traffic and discuss scalability.

Across the stages, tcpdump will collect traffic and Wireshark will analyse captures and generate graphs. Other sources may also provide experimental evidence. The goal is to observe, measure, compare, and interpret, beyond simply checking whether each solution works.

4. Preparing the experimental environment

The multimedia content, CORE topology, and required auxiliary scripts will be supplied through e-learning. This preparation ensures that all groups start with a functional, comparable environment and spend less time on technical setup.

Note: Complete this preparation before the first practical class where possible. Resolve installation, resource access, or script execution problems before starting experimental measurements.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

4.1. Provided resource package

All material needed to prepare and run TP1 is provided in a single package. Extract it so that the resource directory is located at ~/esr/TP1, with the following structure:

```
~/esr/TP1/
├── topology/
│   ├── tp1.xml
│   ├── CORE-VERSION.txt
│   ├── README-TOPOLOGY.md
│   └── topology-reference.png
├── media/
│   ├── MEDIA-INFO.txt
│   ├── video_ref.mp4
│   ├── video_360p_500k.mp4
│   ├── video_540p_1000k.mp4
│   ├── video_720p_2000k.mp4
│   └── video_rtp.mp4
├── dash/
│   ├── prepare_dash.sh
│   ├── start-dash-server.sh
│   ├── open-dash-player.sh
│   ├── player.html
│   ├── dash.all.min.js
│   └── content/              (generated by prepare_dash.sh)
├── http23/
│   ├── start-http2-server.sh
│   ├── start-http3-server.sh
│   ├── get-http2.sh
│   ├── get-http3.sh
│   └── certs/
│       ├── tp1.crt
│       └── tp1.key
├── multicast/
│   ├── enable-multicast.sh
│   └── README.txt
├── rtp/
│   └── README.txt
├── scripts/
│   ├── check-media.sh
│   ├── check-package.sh
│   └── tp1-env.sh
├── tools/
│   └── curl-http3/bin/curl
└── results/                (group captures, logs, and results)
```

Example package installation:
mkdir -p ~/esr
unzip ESR-TP1-26-27-resources.zip -d ~/esr

The supplied scripts standardise environment setup and reduce the time spent on technical configuration. Their implementation is not assessed, but students are encouraged to inspect them to understand the commands and parameters used.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

The file video_ref.mp4 contains the reference multimedia content. The three MPEG-DASH representations — video_360p_500k.mp4, video_540p_1000k.mp4, and video_720p_2000k.mp4 — contain the same content on the same timeline, encoded at different resolutions and bitrates. video_rtp.mp4 is prepared for the RTP/RTCP experiments. All supplied videos include a synthetic reference AAC-LC audio track. In the three DASH representations, audio is shared across video qualities and packaged as a separate Adaptation Set.

The following artefacts are generated during TP1 and are not supplied in advance:

- dash/content/video_manifest.mpd and the MPEG-DASH segments, generated by prepare_dash.sh;
- rtp/pc1.sdp, rtp/pc2.sdp, rtp/pc3.sdp, and rtp/pc4.sdp, generated by FFmpeg in the RTP unicast experiments;
- rtp/multicast.sdp, generated by FFmpeg in the RTP multicast experiment;
- .pcap files, tables, and other experimental data produced by the group.

Note: Do not change the directory structure during TP1. In WSL, the package is at ~/esr/TP1. In CORE/VCMD terminals, which normally run as root, ~ may refer to /root; after loading tp1-env.sh, use $TP1_ROOT to access the package.

4.2. Experimental topology

The same base topology is used for all three stages. This allows experiments to run on a common infrastructure and makes their results easier to compare.

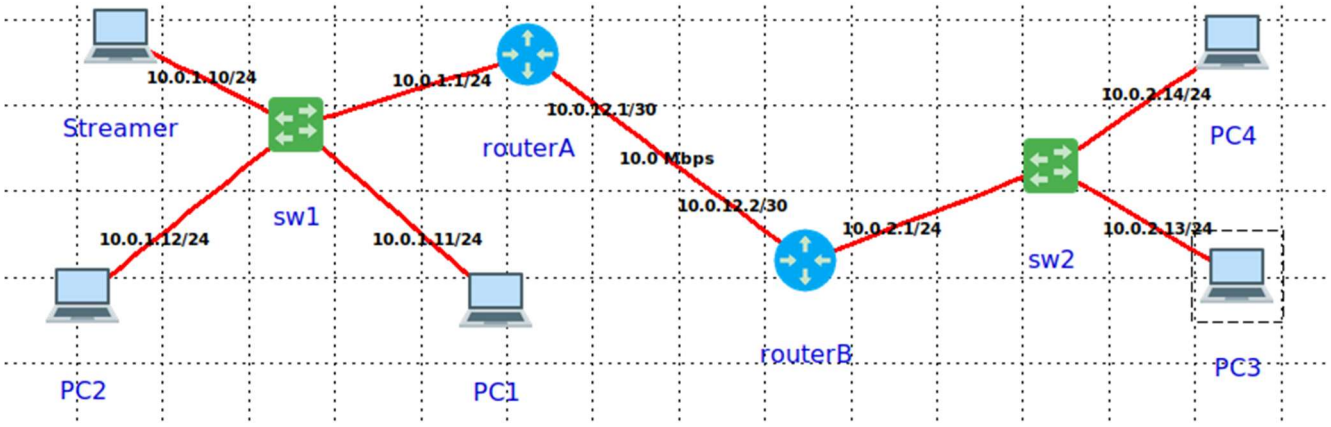


Figure 1 — TP1 base topology.

The topology contains:

- **Streamer — content server used in all three stages;**
- **PC1 and PC2 — clients connected to the access network associated with switch sw1;**
- **PC3 and PC4 — clients connected to the access network associated with switch sw2;**
- **routerA and routerB — routers connecting the two access networks;**

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

- **routerA–routerB — main test link, where bandwidth, delay, and packet loss are changed in a controlled manner.**

Unless otherwise stated, PC3 is the reference client in Stages 1 and 2. All four clients are used in Stage 3.

The package includes the auxiliary configuration needed to enable multicast forwarding between the networks. In Stage 3, enable it with enable-multicast.sh. Students are not expected to configure multicast routing protocols; the aim is to analyse multicast's effect on network traffic.

4.3. Environment validation

1. Start WSL2/Ubuntu and CORE Emulator.

```
sudo core-gui
```

2. In CORE, open ~/esr/TP1/topology/tp1.xml and start the emulation session.

3. Identify Streamer, PC1–PC4, routerA, routerB, and the routerA–routerB test link in the topology.

4. Load the supplied environment variables in every shell used during TP1.

```
source "$(find /home -maxdepth 5 -type f -path '*/esr/TP1/scripts/tp1-env.sh' -print -
quit)"
export GROUP=PL14   # replace PL14 with your group identifier
```

5. From Streamer, verify IP connectivity to PC3 and inspect the routing table. PC3's address is stored in $PC3_IP.

```
ping -c 4 $PC3_IP
ip route
```

6. Repeat the connectivity test for PC1, PC2, and PC4. All clients must be reachable before proceeding.

7. Analyse the reference multimedia file with ffprobe.

```
ffprobe -hide_banner $TP1_ROOT/media/video_ref.mp4
```

Record at least the codec, resolution, frame rate, duration, and approximate bitrate.

8. Play the file locally and confirm that its content is intact.

```
ffplay $TP1_ROOT/media/video_ref.mp4
```

9. In CORE, open Wireshark on the routerA–routerB link. Briefly capture a ping between Streamer and PC3 and confirm that ICMP packets are visible.

```
icmp
```

10. Save this test capture if desired. It need not be included in the report unless it adds useful information to the analysis.

Note: Whenever you change a link's bandwidth, delay, or packet loss, record the applied values. Include them in the report as evidence of the conditions under which the experiments were run.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

Traffic capture with tcpdump

In the following experiments, capture traffic using tcpdump in the terminal of the specified node. Analyse the resulting files later in Wireshark.

Before capturing traffic, identify the available interfaces:

```
ip -br addr
```

For example, to capture all traffic on eth0:

```
tcpdump -i eth0 -s 0 -w /tmp/captura.pcap
```

The -s 0 option captures complete packets. Stop the capture with Ctrl+C.

tcpdump writes PCAP files that Wireshark can open directly. To save a capture in the results directory:

```
mv /tmp/captura.pcap $TP1_ROOT/results/captura.pcap
```

The PCAP file can then be opened in Wireshark:

```
cd "$TP1_ROOT/results"
explorer.exe .
wireshark captura.pcap &
```

Where possible, use a tcpdump capture filter to limit the file to traffic relevant to the experiment. Note that tcpdump capture filters and Wireshark display filters use different syntax.

4.4. Experimental profile assigned to the group

In addition to the reference scenario common to all groups, each group uses one experimental profile. The last digit of the group identifier determines the profile:

GROUP IDS ENDING IN	PROFILE
0 or 1	A
2 or 3	B
4 or 5	C
6 or 7	D
8 or 9	E

The parameters are:

PROFILE	BANDWIDTH	CORE DELAY (ONE WAY)	LOSS
A	4 Mbit/s	10 ms	2%
B	10 Mbit/s	80 ms	2%
C	10 Mbit/s	10 ms	1%
D	10 Mbit/s	80 ms	1%
E	4 Mbit/s	80 ms	1%

Store the profile letter in a shell variable to help identify the files produced. For example, for Profile C:

```
export PROFILE=C
```


© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

To apply a profile, select the routerA–routerB link in CORE and set the bandwidth, delay, and loss properties to the values in the table.

After applying the profile, use ping between PC3 and Streamer to measure and record the observed mean RTT.

```
ping -c 10 $STREAMER_IP
```

Note: In CORE, Delay is the configured one-way delay on the link. With Symmetric Link Effects, a 10 ms Delay (10000 µs in the GUI) contributes approximately 20 ms to the RTT. Always use the RTT actually measured by ping in the analysis.

Suggested division of work for groups of three: during measurements, one student controls network settings in CORE, another controls the application/client, and the third monitors Wireshark and records results. Rotate roles between experiments if useful.

4.5. Simple HTTP baseline

Before the main experiments, transfer a file over simple HTTP between Streamer and PC3. This short baseline validates the data path, identifies the HTTP/TCP stack, and establishes an initial traffic reference.

1. Reset the routerA–routerB link to 10 Mbit/s, 10 ms delay, and 0% loss.

2. On Streamer, enter the directory containing the test file and start a simple HTTP server.

```
cd $TP1_ROOT/media
python3 -m http.server 8080 --bind 0.0.0.0
```

3. On PC3, verify server access and download the test file.

```
curl -I http://$STREAMER_IP:8080/video_ref.mp4
curl -o /tmp/http_baseline.mp4 http://$STREAMER_IP:8080/video_ref.mp4
```

4. On PC3, start an HTTP traffic capture:

tcpdump -i eth0 -s 0 -w /tmp/http_baseline.pcap "tcp port 8080 and host $STREAMER_IP"

5. While the capture is running, open another terminal on PC3 and repeat the transfer:

curl -o /tmp/http_baseline.mp4 http://$STREAMER_IP:8080/video_ref.mp4

When the transfer finishes, return to the tcpdump terminal and stop the capture with Ctrl+C.

6. Save the capture in the results directory:

mv /tmp/http_baseline.pcap $TP1_ROOT/results/${GROUP}_HTTP_baseline.pcap

Later, open the file in Wireshark and use, for example, this filter:

tcp.port == 8080

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

Identify the IP/TCP/HTTP stack in the capture and use Statistics > I/O Graphs to examine throughput over the transfer.

7. Confirm that the received file has the same size as the original; optionally verify its hash.

```
ls -lh $TP1_ROOT/media/video_ref.mp4 /tmp/http_baseline.mp4
sha256sum $TP1_ROOT/media/video_ref.mp4 /tmp/http_baseline.mp4
```

Minimum report evidence: a Wireshark capture or figure showing the HTTP/TCP stack and traffic information useful for analysis. It must demonstrate execution and provide a reference for later comparisons.

5. Stage 1 — Adaptive streaming over HTTP (MPEG-DASH)

This stage examines adaptive streaming over HTTP using MPEG-DASH (Dynamic Adaptive Streaming over HTTP). The same content is supplied in representations with different resolutions and bitrates, allowing the client to adapt playback to network conditions.

The experimental aim is to relate:

network conditions → effective throughput → selected representation → quality of experience

5.1. Preparing MPEG-DASH content

Three representations of the same multimedia content are provided. They have the same duration and, as far as possible, the same codec, frame rate, and temporal alignment; their main differences are resolution and video bitrate. For clarity, they are labelled R1, R2, and R3:

REP.	FILE	RESOLUTION	BITRATE
R1	video_360p_500k.mp4	360p	500 kbit/s
R2	video_540p_1000k.mp4	540p	1.0 Mbit/s
R3	video_720p_2000k.mp4	720p	2.0 Mbit/s

1. Analyse each representation with ffprobe.

```
ffprobe -hide_banner $TP1_ROOT/media/video_360p_500k.mp4
ffprobe -hide_banner $TP1_ROOT/media/video_540p_1000k.mp4
ffprobe -hide_banner $TP1_ROOT/media/video_720p_2000k.mp4
```

2. Record the resolution, codec, frame rate, duration, and approximate bitrate for each representation.

3. Prepare the DASH content using the supplied script.

```
cd $TP1_ROOT/dash
chmod +x prepare_dash.sh start-dash-server.sh open-dash-player.sh
./prepare_dash.sh
```

Note: The script uses the three representations in $TP1_ROOT/media and generates the MPD and DASH segments in $TP1_ROOT/dash/content. Its output summarises the numbers of representations and segments. Students need not implement the script, but should inspect it and understand the commands it runs.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

4. Confirm that the MPD and segments were created.

```
ls -lh $TP1_ROOT/dash/content
```

5. Inspect the MPD and find the available representations.

```
grep -E 'Representation|bandwidth|width|height'
$TP1_ROOT/dash/content/video_manifest.mpd
```

6. Record the identifier, resolution, and bandwidth attribute of each representation and compare them with the ffprobe values.

5.2. Starting the service and validating the player

1. Start the CORE topology and verify connectivity between Streamer and PC3.

2. On Streamer, start the supplied DASH server. It serves the player, MPD, and segments over HTTP/1.1 and disables caching and conditional responses, ensuring that every segment traverses the test link and is visible in Wireshark.

```
cd $TP1_ROOT/dash
./start-dash-server.sh
```

3. On PC3, verify that the MPD is accessible over the network.

```
curl -I http://$STREAMER_IP:8080/content/video_manifest.mpd
```

4. Start the DASH player with the supplied script. The script launches a clean Firefox session for each trial and automatically handles WSLg audio configuration when available.

```
cd $TP1_ROOT/dash
./open-dash-player.sh PC3
```

Note: The script opens the supplied player.html on PC3, configured to use the MPD served by Streamer. You are not expected to configure X11, browser profiles, PulseAudio/WSLg, or JavaScript libraries manually. The player uses automatic ABR with a reduced target buffer of approximately 8–10 s so that transitions can be observed during the experiment.

5. Confirm playback of the reference video and audio. In the player, identify R1/R2/R3, resolution, nominal bitrate, estimated throughput, buffer, and actual rebuffering count.

6. Open Wireshark on the routerA–routerB link and filter by the HTTP server port.

```
tcp.port == 8080
```

5.3. Reference scenario

Configure the routerA–routerB link as follows:

PARAMETER	VALUE
Bandwidth	10 Mbit/s
CORE delay (one way)	10 ms
Packet loss	0%

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

1. Stop any previous playback, start a new Wireshark capture, and run open-dash-player.sh PC3 again. The script creates a clean Firefox profile for the new trial.

2. Play for at least 60 seconds. In the reference scenario, allow the representation and buffer to stabilise.

3. Record the representation selected initially and after playback stabilises, together with the throughput and buffer shown by the player.

4. In Wireshark, use Statistics > I/O Graphs with an interval of approximately one second to observe throughput over time.

5. Determine and record a representative mean throughput during a stable portion of playback.

6. Identify the protocol stack used to transport DASH segments.

7. Save the capture using the standard filename.

```
$TP1_ROOT/results/${GROUP}_E1_baseline.pcap
```

Record the results:

SCENARIO	CONFIG. BW	RTT	LOSS	REPRESENTATION	THROUGHPUT
Reference					

5.4. Dynamic bandwidth variation experiment

This experiment is intended to trigger representation changes during one playback session. Keep delay and loss constant and change only the bandwidth of the routerA–routerB link.

INTERVAL	AVAILABLE BANDWIDTH
0–30 s	10 Mbit/s
30–60 s	2 Mbit/s
60–90 s	800 kbit/s
90–120 s	5 Mbit/s

1. Restore the initial settings: 10 Mbit/s, 10 ms, and 0% loss.

2. Prepare the results table. One group member should control CORE network settings while another monitors the player and Wireshark.

3. Start a Wireshark capture and a new player session. The player starts an experiment clock at T+0 s and separately tracks video time.

4. At 30 s, 60 s, and 90 s, change only the bandwidth to the values in the table. Immediately after clicking Apply in CORE, click the corresponding button in the player to mark the network condition in the log.

5. The player automatically logs R1/R2/R3 changes and rebuffering events during playback. Observe the time between each network change and the representation change.

6. End the experiment after at least 120 s, save the capture, and use the player's Save log button to preserve the event log.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

```
$TP1_ROOT/results/${GROUP}_E1_dinamico.pcap
```

7. Analyse the Wireshark throughput graph and player log, relating bandwidth changes to throughput, buffer, selected representation, and any rebuffering.

INTERVAL	CONFIG. BW	THROUGHPUT	REPRESENTATION	ABR CHANGE TIME	INTERRUPTION?
0–30 s	10 Mbit/s				
30–60 s	2 Mbit/s				
60–90 s	800 kbit/s				
90–120 s	5 Mbit/s				

5.5. Experiment with the assigned group profile

1. Consult the group's assigned Profile A–E in Section 4 and configure the routerA–routerB link accordingly.

2. Measure and record the mean observed RTT between PC3 and Streamer.

```
ping -c 10 $STREAMER_IP
```

3. Restart the player using open-dash-player.sh PC3, start a new Wireshark capture, and play for at least 60 seconds.

4. Record the selected representation, observed throughput, buffer, and any rebuffering. Save the player log if it helps the analysis.

5. Compare these results directly with the reference scenario. Do not change other parameters during the comparison.

6. Save a representative capture.

```
$TP1_ROOT/results/${GROUP}_E1_perfil_${PROFILE}.pcap
```

5.6. Evidence and questions

The report must contain proof of execution and enough experimental evidence to support the discussion. For this stage, include at least:

- an MPD excerpt or screenshot identifying the representations;
- a figure, table, or graph showing the relationship between available/observed throughput and selected representation;
- evidence from the dynamic experiment showing representation changes and marked times of bandwidth changes; the exported player log may be used;
- a quantitative comparison between the reference scenario and the group's experimental profile.

Interpret the evidence. Avoid screenshots that merely show commands already described in these instructions.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

Question 1. How does available bandwidth relate to the representation selected by the client? Support your answer with measured values.

Question 2. Does the representation change immediately when network conditions change? Describe the observed behaviour and offer a possible explanation.

Question 3. What is the role of the MPD in MPEG-DASH? Identify information in this MPD that is relevant to adaptation.

Question 4. Compare the reference scenario with the assigned group profile. What differences did you observe in throughput, selected representation, and quality of experience?

Question 5. Does a representation's nominal bitrate exactly match the throughput observed on the network? Explain any differences.

Question 6. Which conditions caused interruptions or visible playback degradation? Relate your observations to the collected metrics.

6. Stage 2 — HTTP/2 over TCP vs HTTP/3 over QUIC

This stage experimentally compares HTTP/2 over TCP with HTTP/3 over QUIC. Students will run two preconfigured solutions under the same network conditions and measure how their protocol stacks affect transfer time and throughput.

Both solutions transfer $TP1_ROOT/media/video_ref.mp4 over the same topology under the same network settings. The get-http2.sh and get-http3.sh scripts save the downloaded copies as /tmp/http2_video_ref.mp4 and /tmp/http3_video_ref.mp4, respectively. Start a new client process for every measurement to avoid reusing a cache or previous connection.

6.1. Preparing the experiment

Four scripts in $TP1_ROOT/http23 standardise server startup and transfers. start-http2-server.sh starts nginx for HTTP/2; start-http3-server.sh starts Caddy for HTTP/3; get-http2.sh transfers the file using system curl; and get-http3.sh uses the package's curl build with QUIC/HTTP/3 support. Students need not install, compile, or configure these applications manually. The supplied scripts select the necessary versions and options. Inspect them to identify the applications and settings used.

1. Start the topology and verify Streamer–PC3 connectivity.

2. Reset the routerA–routerB link to 10 Mbit/s, 10 ms, and 0% loss.

3. On Streamer, open two terminals and start the HTTP/2 and HTTP/3 servers.

```
cd $TP1_ROOT/http23
./start-http2-server.sh
./start-http3-server.sh
```

4. On PC3, verify that you can download the same file using both solutions.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

```
cd $TP1_ROOT/http23
./get-http2.sh
./get-http3.sh
```

5. Confirm that both received files are identical to the original.

```
ls -lh $TP1_ROOT/media/video_ref.mp4 /tmp/http2_video_ref.mp4 /tmp/http3_video_ref.mp4
sha256sum $TP1_ROOT/media/video_ref.mp4 /tmp/http2_video_ref.mp4
/tmp/http3_video_ref.mp4
```

6. Make a short capture of each protocol on PC3. For HTTP/2, start:

```
tcpdump -i eth0 -s 0 -w /tmp/http2_test.pcap "host $STREAMER_IP and
tcp port $HTTP2_PORT"
```

While the capture is running, execute in another terminal:

```
$TP1_ROOT/http23/get-http2.sh
```

Stop the capture with Ctrl+C. Repeat for HTTP/3:

```
tcpdump -i eth0 -s 0 -w /tmp/http3_test.pcap "host $STREAMER_IP and
udp port $HTTP3_PORT"
```

and, in another terminal:

```
$TP1_ROOT/http23/get-http3.sh
```

Stop the capture again. The PCAP files can be opened directly in Wireshark.

```
cd /tmp
explorer.exe .
wireshark /tmp/http2_test.pcap &
wireshark /tmp/http3_test.pcap &
```

Open the files in Wireshark. For HTTP/2, use tcp.port == 8443, for example. For HTTP/3, use quic || udp.port == 8444.

Note: HTTP content decryption is not required. Identify the differences between the HTTP/2 → TLS → TCP → IP and HTTP/3 → QUIC → UDP → IP stacks, as well as connection setup and evolution.

As further evidence, inspect connection establishment and the Application-Layer Protocol Negotiation (ALPN) extension. In Wireshark, use tls.handshake.extensions_alpn_str. An HTTP/2 connection should reveal h2; the handshake carried over QUIC in HTTP/3 should reveal h3. ALPN confirms that the intended application protocol was negotiated.

6.2. Reference scenario

1. With routerA–routerB set to 10 Mbit/s, 10 ms one-way delay, and 0% loss, measure the RTT using 10 ICMP messages and record the observed mean.

```
ping -c 10 $STREAMER_IP
```

2. Remove files from previous measurements:

© Flávio de Oliveira Silva - Department of Informatics, University of Minho – flavio@di.uminho.pt – For academic use only. External redistribution is prohibited.

```
rm -f /tmp/http2_video_ref.mp4 /tmp/http3_video_ref.mp4
```

3. Run three independent HTTP/2 transfers, deleting /tmp/http2_video_ref.mp4 between runs, and record the three times:

```
/usr/bin/time -f "tempo=%e s" $TP1_ROOT/http23/get-http2.sh
```

4. Repeat exactly the same procedure for HTTP/3, deleting /tmp/http3_video_ref.mp4 between runs:

```
/usr/bin/time -f "tempo=%e s" $TP1_ROOT/http23/get-http3.sh
```

5. Calculate the mean of three runs and the mean throughput for each protocol. If needed, use:

```
mean throughput [bit/s] = (8 × file size [bytes]) / time [s]
```

6. Finally, perform another representative transfer of each protocol while capturing traffic. For HTTP/2, start:

```
tcpdump -i eth0 -s 0 -w /tmp/E2_baseline_h2.pcap "host $STREAMER_IP and tcp port
$HTTP2_PORT"
```

run $TP1_ROOT/http23/get-http2.sh in another terminal and stop the capture with Ctrl+C. For HTTP/3, start:

```
tcpdump -i eth0 -s 0 -w /tmp/E2_baseline_h3.pcap "host $STREAMER_IP and udp port
$HTTP3_PORT"
```

run $TP1_ROOT/http23/get-http3.sh in another terminal and stop the capture with Ctrl+C.

7. Save the captures in the results directory:

```
mv /tmp/E2_baseline_h2.pcap $TP1_ROOT/results/${GROUP}_E2_baseline_h2.pcap
mv /tmp/E2_baseline_h3.pcap $TP1_ROOT/results/${GROUP}_E2_baseline_h3.pcap
```

6.3. Experiment with the assigned group profile

1. Apply the group's assigned Profile A–E to the routerA–routerB link.

2. Measure and record RTT again using ping. Also confirm the configured bandwidth and loss in CORE.

3. Transfer the same file three times over HTTP/2, deleting the output file between runs and starting a new client each time.

4. Run three HTTP/3 transfers under the same conditions.

5. Calculate the mean transfer time and throughput for each protocol.

6. Capture one additional transfer of each protocol using tcpdump, following the reference scenario procedure. Use /tmp/E2_perfil_h2.pcap and /tmp/E2_perfil_h3.pcap as temporary capture files.

7. Save the captures in the results directory with these names:

```
mv /tmp/E2_perfil_h2.pcap $TP1_ROOT/results/${GROUP}_E2_perfil_${PROFILE}_h2.pcap
mv /tmp/E2_perfil_h3.pcap $TP1_ROOT/results/${GROUP}_E2_perfil_${PROFILE}_h3.pcap
```


© Flávio de Oliveira Silva - Department of Informatics, University of Minho – flavio@di.uminho.pt – For academic use only. External redistribution is prohibited.

Note: A valid comparison requires HTTP/2 and HTTP/3 to run under equivalent network conditions. Do not change any other parameter between them.

6.4. Evidence, recording, and analysis of results

SCENARIO	PROTOCOL	BW	RTT	LOSS	TIME	THROUGHPUT
Reference	HTTP/2/TCP					
Reference	HTTP/3/QUIC					
Profile	HTTP/2/TCP					
Profile	HTTP/3/QUIC					

The report must include at least:

- a representative HTTP/2 capture identifying TCP, TLS, and HTTP/2, including ALPN h2 negotiation where visible;
- a representative HTTP/3 capture identifying UDP and QUIC, including ALPN h3 negotiation; evidence of connection setup differences, especially TCP establishment for HTTP/2 and direct UDP/QUIC use for HTTP/3;
- a table of mean times and throughputs for the reference scenario and group profile;
- a figure or graph derived from the captures that helps discuss the effect of network conditions on at least one solution.

Question 1. Identify and compare the protocol stacks used by HTTP/2 and HTTP/3. What differences can you observe in Wireshark captures?

Question 2. Compare HTTP/2/TCP and HTTP/3/QUIC in the reference scenario. Are there meaningful differences in mean transfer time and mean throughput?

Question 3. Compare the reference scenario with the group profile. Which parameter or combination appears to have the greatest effect on the observed behaviour?

Question 4. Based on your evidence, explain why HTTP/3/QUIC cannot be assumed to perform better in every situation. Under what conditions might the observed differences matter for a multimedia service?

Stage 3 — RTP/RTCP over UDP: unicast vs multicast

This stage examines distribution of a real-time multimedia stream using RTP over UDP and compares the cost of unicast and multicast. The experiment uses the same base topology and directly shows where traffic duplication occurs.

Multicast traverses routerA and routerB using the configuration provided in the TP1 package. Configuring multicast routing protocols is outside the scope; the focus is RTP distribution, traffic replication, and scalability.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

Preparing the experiment

1. Restore the base topology and verify unicast connectivity from Streamer to PC3 and PC4.

2. In every new CORE shell used in this stage, first load the TP1 variables:

```
source "$(find /home -maxdepth 5 -type f -path '*/esr/TP1/scripts/tp1-env.sh' -print -quit)"
```

Note: This command locates and loads tp1-env.sh, automatically defining $TP1_ROOT in the CORE shell.

3. Enable multicast configuration on each router separately. Open a shell on routerA and run:

```
sudo $TP1_ROOT/multicast/enable-multicast.sh
```

Then open a shell on routerB and run the same command:

```
sudo $TP1_ROOT/multicast/enable-multicast.sh
```

Note: Multicast forwarding configuration is not assessed. enable-multicast.sh is supplied to enable or verify the necessary configuration consistently on both routers.

4. Analyse the RTP video and record its approximate bitrate:

```
ffprobe -hide_banner $TP1_ROOT/media/video_rtp.mp4
```

5. Capture traffic using tcpdump in CORE shells and analyse it later in Wireshark on Windows. To observe the routerA–routerB link, use the interface on routerA connected to routerB (eth1 in the supplied topology). Before capturing, verify the interface with ip addr.

6. If Wireshark does not automatically identify a UDP flow as RTP, select a packet in the flow and use Analyze > Decode As... > RTP. With two unicast streams, you may need to apply Decode As to each UDP flow separately.

7. To inspect RTP fields, use Telephony > RTP > RTP Streams and examine Payload Type, Sequence Number, Timestamp, and SSRC, among others.

8. Before each scenario, stop the previous streams and start a fresh capture to avoid mixing traffic from different scenarios.

Unicast experiment

Run two scenarios with the same content and bitrate per stream.

Scenario U1 — PC3 only

1. On Streamer, start an RTP stream to PC3 and generate its SDP file:

```
cd $TP1_ROOT/rtp
ffmpeg -re -stream_loop -1 -i $TP1_ROOT/media/video_rtp.mp4 -an -c:v copy -f rtp \
  -sdp_file pc3.sdp rtp://$PC3_IP:5004
```


© Flávio de Oliveira Silva - Department of Informatics, University of Minho – flavio@di.uminho.pt – For academic use only. External redistribution is prohibited.

2. On PC3, ensure graphical applications can use the WSL/Windows display and start playback from the SDP file:

```
export DISPLAY=:0
ffplay -protocol_whitelist file,udp,rtp $TP1_ROOT/rtp/pc3.sdp
```

3. On routerA, start a capture on the routerA–routerB link:

```
tcpdump -i eth1 -s 0 -w /tmp/E3_U1_PC3.pcap "udp and host $STREAMER_IP"
```

4. Run the experiment for approximately 30 s, stop the capture with Ctrl+C, and record the observed throughput. Save the capture:

```
mv /tmp/E3_U1_PC3.pcap $TP1_ROOT/results/${GROUP}_E3_U1_PC3.pcap
```

Scenario U2 — PC3 + PC4

5. Keep the PC3 stream running, open another terminal on Streamer, and start a second independent unicast stream to PC4:

```
cd $TP1_ROOT/rtp
ffmpeg -re -stream_loop -1 -i $TP1_ROOT/media/video_rtp.mp4 -an -c:v copy -f rtp \
  -sdp_file pc4.sdp rtp://$PC4_IP:5004
```

6. On PC4, start playback:

```
export DISPLAY=:0
ffplay -protocol_whitelist file,udp,rtp $TP1_ROOT/rtp/pc4.sdp
```

7. On routerA, start a new capture on the routerA–routerB link:

```
tcpdump -i eth1 -s 0 -w /tmp/E3_U2_PC3_PC4.pcap "udp and host $STREAMER_IP"
```

8. Run for approximately 30 s, stop the capture with Ctrl+C, and confirm in Wireshark that two independent RTP streams target PC3 and PC4. Record the aggregate link traffic.

```
mv /tmp/E3_U2_PC3_PC4.pcap $TP1_ROOT/results/${GROUP}_E3_U2_PC3_PC4.pcap
```

Multicast experiment

Stop all unicast streams before starting. A single RTP stream will be sent to multicast group 239.10.10.10.

1. Confirm that multicast forwarding has been enabled on routerA and routerB as described in the preparation.

2. On Streamer, start a single multicast stream and generate its SDP file:

```
cd $TP1_ROOT/rtp
ffmpeg -re -stream_loop -1 -i $TP1_ROOT/media/video_rtp.mp4 -an -c:v copy -f rtp \
  -sdp_file multicast.sdp "rtp://239.10.10.10:5004?ttl=16"
```

3. On PC3, start playback:

```
export DISPLAY=:0
ffplay -protocol_whitelist file,udp,rtp $TP1_ROOT/rtp/multicast.sdp
```


© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

4. Without restarting Streamer, run the same command on PC4. Confirm that PC3 and PC4 simultaneously receive the same multicast stream.

```
export DISPLAY=:0
ffplay -protocol_whitelist file,udp,rtp $TP1_ROOT/rtp/multicast.sdp
```

5. On routerA, capture approximately 30 s on the routerA–routerB link:

```
tcpdump -i eth1 -s 0 -w /tmp/E3_multicast_PC3_PC4.pcap "udp and dst host 239.10.10.10"
```

6. Stop the capture with Ctrl+C and open the file in Wireshark. Confirm that the destination address is 239.10.10.10 and that only one RTP stream crosses the routerA–routerB link despite having two receivers.

```
ip.dst == 239.10.10.10
```

7. Save a representative capture:

```
mv /tmp/E3_multicast_PC3_PC4.pcap $TP1_ROOT/results/${GROUP}_E3_multicast_PC3_PC4.pcap
```

Note: Because Wireshark is installed on Windows, you can open the TP1 directory from WSL with explorer.exe . and open .pcap files directly in Wireshark. Installing the Wireshark GUI inside WSL is unnecessary.

Evidence and scalability analysis

For this stage, the report must provide evidence of execution and support the unicast/multicast comparison. Include at least:

- an RTP capture from U1 identifying source/destination addresses and relevant RTP fields;
- a U2 capture showing two independent RTP streams on the routerA–routerB link;
- a multicast RTP capture identifying group address 239.10.10.10 and showing a single stream on the routerA–routerB link;
- a table comparing observed routerA–routerB link throughput in U1, U2, and multicast with two receivers.

Let R be the mean bitrate of one stream and N the number of clients. Compare measurements with these approximations:

unicast: $B_{server} \approx N \times R$ multicast: $B_{server} \approx R$

Using the measured R, estimate server output traffic for N = 10, N = 100, and N = 1000 unicast clients and compare it conceptually with multicast.

Question 1. How did traffic on the routerA–routerB link change from one to two unicast streams? Do the results approximate N × R?

Question 2. In U2, how can you distinguish the two independent RTP streams in Wireshark? Use addresses, ports, and/or SSRC in your answer.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

Question 3. How did multicast link traffic change when the second receiver joined? Where does the replication needed to deliver the stream to both clients occur?

Question 4. What information does the RTP header provide in this experiment? Identify Payload Type, Sequence Number, Timestamp, and SSRC in the captures. If you observe RTCP traffic, explain its role too.

Question 5. If multicast reduces duplicate traffic, why can a global Internet content distribution service not simply assume end-to-end IP multicast? Relate your answer to the motivation for application-layer distribution architectures.

7. Report and submission

The report must clearly and concisely present experimental settings, proof of execution, results, and analysis. A step-by-step retelling of these instructions is not required; focus on the evidence and what it shows.

7.1. Identification and suggested structure

The first page of the report must show:

- practical class group (PL);
- group identifier (e.g., PL10);
- names and student numbers of all three students;
- assigned experimental profile (A–E).

Suggested structure:

- Experimental setup — topology, group profile, and relevant parameters;
- HTTP baseline — evidence of the transfer and identification of the HTTP/TCP stack;
- Stage 1 — MPEG-DASH results, dynamic experiment, and group profile;
- Stage 2 — HTTP/2/TCP vs HTTP/3/QUIC comparison;
- Stage 3 — RTP unicast vs multicast comparison and scalability analysis;
- Discussion — comparison across stages and main trade-offs observed;
- Conclusions — main findings supported by experimental data.

7.2. Proof of execution and experimental evidence

The report must show that the experiments were performed. Where possible, select evidence that both demonstrates execution under the specified conditions and provides useful data for analysis.

Possible evidence includes:

- Wireshark traffic captures identifying relevant protocols, flows, addresses, or events;
- Wireshark I/O graphs or traffic statistics;
- relevant excerpts of ffprobe, ffmpeg, ping, or client output;

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.

- information displayed by the DASH player;
- tables of measured values and graphs derived from them;
- topology or link-property screenshots where needed to establish experimental conditions.

Do not include screenshots merely to prove that commands ran if they add no useful information. Evidence must be legible, labelled, and discussed in the text.

Keep .pcap files and other experimental data until assessment is complete. Submit them with the report only if explicitly requested on e-learning.

7.3. Organising evidence files

To make measurements traceable, use filenames containing the group identifier, stage, and scenario. Examples:

PL14_HTTP_baseline.pcap
PL14_E1_baseline.pcap
PL14_E1_perfil_C.pcap
PL14_E2_baseline_h2.pcap
PL14_E2_perfil_h3.pcap
PL14_E3_U2_PC3_PC4.pcap
PL14_E3_multicast_PC3_PC4.pcap

7.4. Submission

Submit the report as a PDF through the group submission facility on the e-learning platform.

Recommended filename:

```
ESR-TP1-PL14.pdf   # example for group PL14
```

Submission deadline: 12 October 2026, by the end of the day.

© Flávio de Oliveira Silva - Department of Informatics, University of Minho — flavio@di.uminho.pt — For academic use only. External redistribution is prohibited.